Research Agent — Zero-Cost Cloud Deployment Plan

Render free web service + Supabase PostgreSQL/pgvector + quota-aware model providers

Prepared: 10 October 2026

Repository: https://github.com/BGRAMPAGE/research_agent

1. Goal and architecture

- Goal: publish both Chainlit profiles at a public HTTPS URL without paying for a server or domain. Zero cost means remaining within free quotas and avoiding paid APIs, overages, and optional services. Free tiers and terms can change.
- Recommended topology: GitHub repository → Render Free Web Service (Python/Chainlit app) → Supabase Free PostgreSQL with pgvector for persistent paper chunks. Use a model provider with a currently available free developer quota, or configure a model endpoint you control. Never assume free quotas are unlimited.
- Do not depend on Render's local filesystem for authoritative paper data, SQLite history, checkpoints, or uploads. Keep durable data in an external database and retain a separate backup.

2. Stack and limitations

- Application: existing Python, Chainlit, LangGraph code. Database: Supabase PostgreSQL + pgvector, if compatible with the repository schema and vector dimension. CI: GitHub Actions. Secrets: Render environment variables and Supabase dashboard.
- Render free web services sleep after 15 minutes of inactivity and may take about a minute to wake. Their local filesystem is ephemeral. This is fine for a demo, not guaranteed production uptime.
- Render free Postgres expires after 30 days, so do not use it as the durable database in this plan. Supabase Free currently lists 500 MB database size and 1 GB file storage; inactive projects may pause. Vector storage can consume quota quickly.
- Long research runs can hit request/proxy timeouts, memory limits, or provider quotas. Limit source count, search iterations, output length, concurrency, and per-user requests. Cloud hosting cannot access laptop localhost/Ollama unless deliberately connected.

3. Prepare the repository

- 1. Confirm a clean clone works locally and push all required files to main.
- 2. Confirm the app can bind to 0.0.0.0 and use the platform-provided PORT. Verify the pinned Chainlit version supports the launch flags.
- 3. Ensure requirements/lock file, .gitignore, .env.example, and deployment README are current.
- 4. Remove hard-coded local Windows paths. Make corpus location, database URL, provider choice, model name, and API keys configurable.
- 5. Make schema creation/migrations repeatable. Do not depend on manually created tables that are undocumented.
- 6. Confirm public error messages do not reveal tracebacks, credentials, local paths, or private prompts.

4. Set up Supabase

- 1. Create a Supabase Free project and securely save the database password.
- 2. In SQL Editor, enable pgvector using the supported extension command, commonly CREATE EXTENSION IF NOT EXISTS vector; confirm extension availability and permissions.
- 3. Apply the repository's actual paper/chunk schema and vector dimension. Do not replace the project's schema with a generic example without reconciling column names.
- 4. Create the appropriate vector index only after checking the supported pgvector version and storage quota. Start with a small corpus.
- 5. Configure the recommended Supabase connection string with SSL. Use a pooled connection if appropriate for the app's connection pattern.
- 6. Ingest 20–50 sample papers/chunks once from your local machine. Verify known queries return expected title, chunk ID, and excerpt. Keep source files backed up locally.

5. Deploy on Render

- 1. Sign in to Render and choose New → Web Service. Connect GitHub and select BGRAMPAGE/research_agent.
- 2. Use the repository root unless the app is actually in a subdirectory. Select a compatible Python runtime.
- 3. Build command: install the actual dependency file, commonly pip install -r requirements.txt if that file exists.

- 4. Start command: use the repository's documented Chainlit launch command. A common Unix-style form is `chainlit run app.py --host 0.0.0.0 --port $PORT`; confirm the pinned Chainlit version and Render shell handle it. If needed, create a start script that reads PORT from the environment.
- 5. Add environment variables from the real config module: database URL, provider selection, model name, hosted model key (if used), search API key (if required), and other required settings. Never put secrets in Git.
- 6. Deploy; inspect build/runtime logs for missing packages, imports, schema mismatches, model download issues, and memory problems.
- 7. Open the public URL and test Deep Research with a short topic and Paper Library with a question whose answer is known from an ingested paper.
- 8. Test an unsupported Paper Library question, citations, cold start after idle sleep, and one or two concurrent sessions. Do not stress-test free quotas.

6. Model and corpus strategy

- Do not download/ingest a large arXiv corpus on every app startup. Ingest once locally against Supabase and store only compact metadata/chunks/vectors in the database.
- Use the same embedding model for stored corpus vectors and query vectors. If changing embedding model or dimension, rebuild the corpus vectors rather than mixing incompatible embeddings.
- If loading BGE-M3 on the host is too slow or memory-heavy, precompute corpus embeddings locally and consider a smaller compatible query embedding model only if the corpus is re-embedded consistently.
- Keep a local source corpus backup. Free database storage is not a backup. Track database size and model-provider quota.

7. Smoke tests and suitability

- Test A: Deep Research returns a report with working source links. Test B: Paper Library returns excerpts and citations for a known question. Negative test: a question absent from the corpus gets an insufficient-evidence response. Failure test: database errors produce a safe message, not a false ‘no results’ answer.
- Suitable: recruiter demo, low-volume personal research, student project, and public-paper experiments. Not suitable as-is for guaranteed uptime, sensitive documents, high concurrency, or high-impact decisions.
- Recommended fallback if quotas are tight: reduce corpus size, cap usage, or use the laptop-hosted plan for private demonstrations rather than risking unexpected costs.

8. Official documentation

- Render free services: https://render.com/docs/free
- Render web services/port binding: https://render.com/docs/web-services
- Supabase free plan and billing: https://supabase.com/docs/guides/platform/billing-on-supabase
- Supabase database size limits: https://supabase.com/docs/guides/platform/database-size

Maintenance note

Before implementation, verify the exact repository commit and re-check provider pricing, free-tier quotas, terms, and security settings. Free-tier limits change.